

3채널 이퀄라이저를 GUI 기반으로 설계하기

오디오 신호처리 12주차 과제

20213032 권영훈

과제 목표:

강의 자료에서 소개한 옥타브/매트랩 용 GUI 설계 방법을 활용하여 3채널 즉, "로우 피킹 - 미디엄 피킹 - 하이 피킹" 이퀄라이저를 GUI 기반으로 설계합니다.

채널 별 게인, Q(품질 팩터)의 초기 설정값은 각각 [0 0 0], [1 1 1] 로 두지만 슬라이더 바를 통해 변경 가능하도록 합니다.

중심 주파수도 초기 설정값($f_s=44100\text{Hz}$ 기준)은 1000Hz(로 피킹), 4000Hz(미디엄 피킹), 7000Hz(하이 피킹) 로 두지만 슬라이더 바를 이용해 변경할 수 있도록 합니다.

슬라이더 바를 통해 설정된 이퀄라이저의 주파수 스펙트럼을 버튼 클릭으로 볼수 있도록 합니다.

설계된 이퀄라이저에 입력 음악 파일을 역시 버튼을 통해 선택할 수 있도록 하고 이퀄라이저 통과 및 출력 신호를 역시 버튼 클릭을 통해 파일로 저장하는 기능도 포함시키기 바랍니다

1. 파이썬 라이브러리 설치:

- !pip install streamlit soundfile scipy matplotlib pyngrok

2. 소스 코드:

```
import numpy as np
import scipy.signal as signal
import matplotlib.pyplot as plt
import librosa
import librosa.display
import soundfile as sf
import ipywidgets as widgets
from IPython.display import HTML, Audio, clear_output
from google.colab import files
# --- 전역 변수 초기화
fs = 44100
audio_data = None
original_audio_path = ''
equalized_path = 'equalized_output.wav'
previous_response = None
equalized_audio = None
```

```

# --- 피킹 EQ 설계
def design_peaking_eq(f0, gain_db, Q, fs):
    10**(gain_db / 40)
    w0 = 2* np.pi * f0 / fs
    alpha = np.sin(w0) / (2* Q)
    b0 = 1+ alpha * A
    b1 = -2* np.cos(w0)
    b2 = 1- alpha * A
    a0 = 1+ alpha / A
    a1 = -2* np.cos(w0)
    a2 = 1- alpha / A
    b = np.array([b0, b1, b2]) / a0
    a = np.array([1, a1 / a0, a2 / a0])
    return b, a
# --- EQ 결합
def make_eq(gains, qs, fcs):
    b, a = [1], [1]
    for g, q, f0 in zip(gains, qs, fcs):
        bi, ai = design_peaking_eq(f0, g, q, fs)
        b = np.convolve(b, bi)
        a = np.convolve(a, ai)
    return b, a
# --- 주파수 응답 시각화
def plot_response(*params):
    global previous_response
    gains = params[:5]
    qs = params[5:10]
    fcs = params[10:]
    b, a = make_eq(gains, qs, fcs)
    w, h = signal.freqz(b, a, worN=1024, fs=fs)
    plt.figure(figsize=(8, 4))
    if previous_response is not None:
        plt.plot(previous_response[0], previous_response[1], label='이전 응답',
        linestyle='--', alpha=0.5)
    plt.plot(w, 20* np.log10(abs(h)), label='현재 응답')
    plt.title("Frequency Response")
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Gain (dB)")
    plt.grid(True)

```

```

plt.legend()
plt.show()
previous_response = (w, 20* np.log10(abs(h)))
# --- EQ 적용 및 저장
def apply_eq_and_save(*params):
    global audio_data, equalized_audio
    if audio_data is None:
        print("먼저 오디오를 업로드하세요.")
        return
    gains = params[:5]
    qs = params[5:10]
    fcs = params[10:]
    b, a = make_eq(gains, qs, fcs)
    y_eq = signal.lfilter(b, a, audio_data)
    equalized_audio = y_eq
    sf.write(equalized_path, y_eq, fs)
    print("    EQ 적용 및 저장 완료")
    display(Audio(equalized_path))
# --- 스펙트로그램 시각화 (선택)
def plot_spectrograms(original, equalized):
    fig, ax = plt.subplots(1, 2, figsize=(12, 4))
    D1 = librosa.amplitude_to_db(np.abs(librosa.stft(original)), ref=np.max)
    D2 = librosa.amplitude_to_db(np.abs(librosa.stft(equalized)), ref=np.max)
    librosa.display.specshow(D1, sr=fs, x_axis='time', y_axis='log', ax=ax[0])
    ax[0].set_title('원본 스펙트로그램')
    librosa.display.specshow(D2, sr=fs, x_axis='time', y_axis='log', ax=ax[1])
    ax[1].set_title('EQ 후 스펙트로그램')
    plt.tight_layout()
    plt.show()
# --- 오디오 업로드
def upload_audio(change):
    global audio_data, original_audio_path
    uploaded = list(upload_btn.value.values())[0]
    original_audio_path = uploaded['metadata']['name']
    with open(original_audio_path, 'wb') as f:
        f.write(uploaded['content'])
    y, _ = librosa.load(original_audio_path, sr=fs)
    audio_data = y
    print(f"업로드 완료: {original_audio_path}")

```

```

display(Audio(y, rate=fs))
# --- 다운로드
def download_eq_file(_):
    files.download(equalized_path)
# --- A/B 비교 버튼
def play_original(_):
    if audio_data is None:
        print("오디오가 없습니다.")
        return
    print("원본 오디오")
display(Audio(audio_data, rate=fs))
def play_equalized(_):
    if equalized_audio is None:
        print("EQ 처리된 오디오가 없습니다. 먼저 EQ를 적용하세요.")
        return
    print("EQ 적용된 오디오")
display(Audio(equalized_audio, rate=fs))
# --- 슬라이더 설정 (5채널)
channels = ['L', 'LM', 'M', 'HM', 'H']
default_fc = [500, 1000, 3000, 5000, 8000]
gain_sliders = [widgets.FloatSlider(value=0, min=-12, max=12, step=0.1,
description=f'Gain {ch}') for ch in channels]
q_sliders = [widgets.FloatSlider(value=1, min=0.1, max=10, step=0.1,
description=f'Q {ch}') for ch in channels]
fc_sliders = [widgets.FloatSlider(value=freq, min=100, max=20000,
step=100, description=f'Fc {ch}(Hz)') for freq, ch in zip(default_fc,
channels)]
# --- 버튼 정의
plot_btn = widgets.Button(description="주파수 응답 보기")
apply_btn = widgets.Button(description="EQ 적용 및 저장")
download_btn = widgets.Button(description="결과 다운로드")
upload_btn = widgets.FileUpload(accept='.wav', multiple=False)
play_original_btn = widgets.Button(description="원본 재생")
play_eq_btn = widgets.Button(description="EQ 후 재생")
# --- 이벤트 연결
plot_btn.on_click(lambda_: plot_response(*(s.value for s in gain_sliders +
q_sliders + fc_sliders)))
apply_btn.on_click(lambda_: apply_eq_and_save(*(s.value for s in gain_sliders
+ q_sliders + fc_sliders)))

```

```

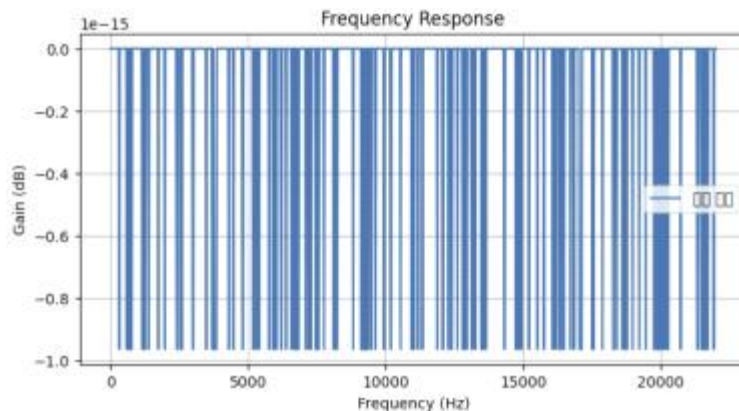
download_btn.on_click(download_eq_file)
upload_btn.observe(upload_audio, names='value')
play_original_btn.on_click(play_original)
play_eq_btn.on_click(play_equalized)
# --- UI 구성
slider_rows = [widgets.HBox([gain_sliders[i], q_sliders[i],
fc_sliders[i]]) for i in range(5)]
gui_layout = widgets.VBox([
widgets.Label("1단계: 오디오 파일 업로드"),
upload_btn,
widgets.Label("2단계: 5채널 EQ 설정 (Gain / Q / Fc)"),
*slider_rows,
widgets.HBox([plot_btn, apply_btn, download_btn]),
widgets.HBox([play_original_btn, play_eq_btn])
])
display(gui_layout)

```

3. 결과 이미지:



3.1 위의 초기 설정값 주파수 응답 이미지:



4. 오디오 파일 업로드 결과:

1단계: 오디오 파일 업로드

Upload (1)

2단계: 5채널 EQ 설정 (Gain / Q / Fc)

Gain L	-6.00	Q L	4.50	Fc L (Hz)	4300.00
Gain LM	2.90	Q LM	2.50	Fc LM (Hz)	3900.00
Gain M	-4.30	Q M	3.70	Fc M (Hz)	9900.00
Gain HM	4.80	Q HM	4.00	Fc HM (Hz)	4100.00
Gain H	-7.40	Q H	7.80	Fc H (Hz)	8000.00

주파수 응답 보기

EQ 적용 및 저장

결과 다운로드

원본 재생

EQ 후 재생

업로드 완료: Silver Scrapes_original.wav

▶ 0:00 / 0:05 🔊 ⋮

✅ EQ 적용 및 저장 완료

▶ 0:00 / 0:05 🔊 ⋮

